

Privacy-Preserving Convolutional Neural Network Classification Scheme With Multiple Keys

Baocang Wang^{ID}, Yange Chen^{ID}, Furong Li^{ID}, Jian Song^{ID}, Rongxing Lu^{ID}, *Fellow, IEEE*, Pu Duan^{ID},
and Zhihong Tian^{ID}, *Senior Member, IEEE*

Abstract—Convolutional Neural Networks (CNNs) possess extensive applicability across diverse domains, particularly in the realm of image recognition. In light of the advent of machine learning as a service, the utilization of a well-trained CNN model by servers to execute image classification based on user queries has become a significant service, catering to a wide array of applications. Nevertheless, this convenience is accompanied by the inherent risk of data privacy and model privacy disclosure, which can have severe ramifications, particularly in the context of specialized scenarios like medical images and location images. Hence, how to perform classification for CNN with privacy protection emerges as a crucial research concern. Furthermore, the nonlinearity of CNN's activation function renders it unsuitable for homomorphic cryptosystems. In order to address these challenges, we put forth a privacy-preserving CNN classification scheme employing a distributed two trapdoors public-key cryptosystem (DT-PKC). Initially, we introduce a security protocol toolkit encompassing protocols for secure multiplication, secure activation function computing, and average pooling. In addition, we propose a novel continuous and derivative Tanhplus function as an approximation of the Relu function, aiming to enhance the accuracy of classification

results. The secure activation function computing protocol utilizes the aforementioned Tanhplus function in conjunction with the proposed homogenization algorithm to compute the activation function. This protocol guarantees more precise and accurate output in the activation function calculation of CNN when operating under ciphertext. Furthermore, the adoption of the DT-PKC cryptosystem not only ensures privacy protection for CNN classification but also provides support for lightweight users and multiple keys. Finally, security analysis and performance evaluations demonstrate that the proposed scheme is secure, practicable, and efficient with high accuracy.

Index Terms—Classification, convolutional neural network, homomorphic encryption, privacy-preserving.

I. INTRODUCTION

NEURAL networks, a prominent area of research in artificial intelligence, have found extensive applications in various domains such as image processing [1], [2], voice recognition [3], medical diagnosis [4], face analysis [5], and neural language processing [6]. Among these, Convolutional Neural Network (CNN), an emerging deep learning architecture, has particularly demonstrated exceptional performance in image recognition, thereby showcasing the promising potential of artificial intelligence applications.

Currently, the exploration of cloud computing [42] and machine learning has given rise to a novel cloud service model known as machine learning as a service [35]. Within this model, the server is capable of offering diverse services, including training and classification, to achieve desired outcomes, occasionally in collaboration with a self-contained assist-server. The model training and classification [12], [13] in neural networks are reliant on extensive data. CNN [41], a significant machine learning algorithm, offers distinct benefits in speech recognition and image processing owing to their unique structure of local weight sharing. This architecture closely resembles the biological neural network and reduces network complexity through weight sharing. Notably, CNNs allow for direct input of multi-dimensional input vectors, refraining from the complexity of data reconstruction in the process of feature extraction and classification.

Due to the numerous benefits associated with neural networks, their applications have significantly enhanced convenience in our daily lives. Nevertheless, there also exist hidden risks of privacy information leakage such as data privacy and model privacy [7], [8], [11]. In certain specific scenarios, the exposure of privacy can lead to severe consequences, particularly in the

Manuscript received 10 January 2022; revised 26 December 2023; accepted 30 December 2023. Date of publication 3 January 2024; date of current version 6 February 2024. This work was supported in part by the Natural Science Foundation of Henan under Grant 232300421392, in part by the National Natural Science Foundations of China under Grants U19B2021 and 62101478, in part by the Key Research and Development Program of Shaanxi under Grant 2020ZDLGY08-04, in part by the China University Innovation Fund under Grant 2022HS063, in part by the Innovation Scientists and Technicians Troop Construction Projects of Henan Province, in part by the Henan International Joint Laboratory of Polarization Sensing and Intelligent Signal Processing, and in part by the National College Students Innovation and Entrepreneurship Training Program under Grant 202310480010. (Corresponding author: Yange Chen; Zhihong Tian.)

Baocang Wang is with the State Key Laboratory of Integrated Service Networks, Xidian University, Xi'an 710071, China (e-mail: bcwang@xidian.edu.cn).

Furong Li is with the 20th Research Institute of China Electronics Technology Group Corporation, Xi'an 710071, China (e-mail: 1551005834@qq.com).

Yange Chen is with the School of Information Engineering, Xuchang University, Xuchang 461000, China, and also with the Henan International Joint Laboratory of Polarization Sensing and Intelligent Signal Processing, Xuchang 461000, China (e-mail: ygchen428@163.com).

Jian Song is with the Wireless and Computing Product R&D Institute, Zhongxing Telecommunications Equipment Corporation, Shenzhen 518057, China (e-mail: soongjian@126.com).

Rongxing Lu is with the Faculty of Computer Science, University of New Brunswick, Fredericton, NB E3B 5A3, Canada (e-mail: rlu1@unb.ca).

Pu Duan is with the Secure Collaborative Intelligence Laboratory, Ant Group, Zhejiang 310000, China (e-mail: p.duan@antgroup.com).

Zhihong Tian is with the Cyberspace Institute of Advanced Technology, Guangzhou University, Guangzhou 510000, China (e-mail: tianzhihong@gzhu.edu.cn).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TSC.2023.3349298>, provided by the authors.

Digital Object Identifier 10.1109/TSC.2023.3349298

case of medical images and location images [10]. Consequently, the primary concern that demands utmost consideration is how to effectively employ neural networks while ensuring privacy protection. For the privacy-preserving CNN classification, there are two principal challenges:

1) The intricate architecture of CNN poses challenges in mitigating privacy breaches and addressing users' online queries and multiple key issues during the classification procedure. Present privacy mechanisms typically employ user-cloud platform interactions for classification. In certain dual-server schemes, one server is authorized to access users' private keys [20], [29], or both servers possess fully-trained models by default [20]. However, these schemes, while ultimately achieving classification, are susceptible to direct exposure of users' private keys or servers' training models to untrusted clouds due to their reliance on low-level security assumptions.

In certain single-server schemes (cloud-as-a-service provider) [15], [16], [25], [30], [33], wherein users engage with the cloud-as-a-service provider, online interaction and collaboration are required to accomplish tasks. However, if users engage in online classification at any given moment, it may lead to superfluous interaction and computational burden. Moreover, the server, possessing a strong private key, can access all input or training data, thereby giving rise to potential security concerns. Furthermore, certain schemes [16], [17], [20] lack the capability to support multiple keys.

2) The activation function plays a crucial role in CNN, but it is not suitable for homomorphic cryptosystems due to its non-linear nature. To address this issue, many existing schemes employ Taylor polynomial approximation [14], [17], [19], [24] or piecewise approximation methods to compute secure activation functions. However, the Taylor expansion polynomial for specific points is inadequate for accurately representing the activation function across its entire domain, while the piecewise function approximation exhibits suboptimal performance in a wide range.

In view of the above challenges, we propose an efficient privacy-preserving CNN classification scheme (PCCS) with multiple keys.

The main contributions are as follows.

- 1) We present a privacy-preserving CNN classification scheme that utilizes the distributed two trapdoors public-key cryptosystem (DT-PKC). By employing the two trapdoors feature of DT-PKC, the classification process is facilitated through the collaboration of two partial strong private keys, thereby safeguarding both data privacy and model privacy.
- 2) We construct the continuous and derivative function called Tanhplus to approximate the Relu function due to the non-derivability of the Relu function at zero point. Besides, we propose a homogenization algorithm (HA) to solve the incorrect results during the activation function calculation to obtain the correct results under ciphertext.
- 3) We design a security protocol toolkit including secure multiplication protocol (SMP), secure activation function computing protocol (SAC), and average pooling protocol (APP). Specifically, the SAC calculates the activation function under ciphertext more accurately and

correctly with the proposed Tanhplus function and HA algorithm.

- 4) We conduct theoretical analysis and experimental evaluations, focusing on privacy protection, accuracy, and efficiency. The results indicate that our scheme has security, practicability, efficiency, and high accuracy while also supporting multiple keys and lightweight users.

II. RELATED WORK

There exist two phases in privacy protection deep learning: the training phase and the classification (prediction) phase. Broadly speaking, the primary objective during the training phase is to construct a model by utilizing a dataset contributed by multiple parties, while guaranteeing the non-disclosure of each user's private data to other participants. Conversely, privacy protection in the classification (prediction) phase aims to securely acquire classification (prediction) outcomes by employing fully-trained parameters. In light of the obstacles encountered in the investigation of privacy-preserving neural networks for classification (prediction) purposes, we categorize them into two distinct domains of research: the privacy-preserving mechanism and the activation function method. In the first area, the authors use different cryptographic or disturbance methods to protect the privacy of neural networks to meet different security requirements [22], [26], [31], [46], while in the second one, the schemes select and deform different activation functions to train or predict in neural networks with homomorphic encryptions [14], [24].

1) *Privacy-Preserving Mechanism*: For the privacy-preserving purpose, Shokri et al. [33] introduced a collaborative training approach for a privacy-preserving neural network, wherein participants can locally train the dataset and subsequently upload updated gradients to a parameter server for further training by other users. While this method yields a more accurate model for multi-user cooperative training, the transmission of a fraction of gradients to the server poses a risk to input privacy, necessitating continuous online interaction with the server by users. Meanwhile, Wang et al. [30] proposed a privacy-preserving neural network training scheme based on Paillier encryption [44] under a single-layer perceptron. In each iteration of the scheme, user sends the activation values to the cloud server for subsequent calculations, which not only risks information leakage during the training process, but also does not support multiple users. To avoid exposure of model parameters to the server, Phong et al. [16] employed Paillier and Learning with Errors (LWE) cryptosystems to encrypt the updated gradients and upload them to the server, enabling multiple users to share the secret key. Unlike [16], Phong et al. [22] substituted the upload gradients as upload weight parameters with the symmetric cryptosystem, while still employing partial local training. Zhang et al. [15] proposed privacy-preserving neural network training schemes with Paillier cryptosystem, one based on the proxy re-encryption and the other on key management. However, their threat model presupposes that all users are honest, which is not sufficient for the general case. Furthermore, their scheme 2 endorses local training, which burdens users with computational overhead. Baryalai et al. [20] constructed a privacy-preserving neural

network classification scheme for dual-cloud servers based on Paillier cryptosystem. However, the scheme allows users to share private keys with the cloud and expose the model parameters to the cloud, which is insecure. Gilad-Bachrach et al. [34] proposed a privacy-preserving CNN classification with a leveled homomorphic encryption (LHE) scheme, but the scheme is only suitable for shallow neural networks. Chabanne et al. [19] addressed the shortcomings of [34] by using a Brakerski-Gentry-Vaikuntanathan (BGV) cryptosystem [45] for prediction, but the fully homomorphism has large ciphertext extension and heavy computational complexity. Liu et al. [26] presented a privacy-preserving prediction scheme with garbled circuit, secret sharing, and homomorphic encryption, but they neglect to protect the neural network output. Hesamifard et al. [17] proposed a novel activation function and used the LHE cryptosystem combined with the single instruction multiple data (SIMD) technique to achieve the privacy-preserving CNN prediction, yet it does not support multiple keys. In [25], the scheme presented a method approximating the Relu function based on the Relu derivative in the training phase and combined it with the batch normalization for ciphertext experiments, realizing both training and classification, but it does not support multiple keys. Mohassel et al. [31] utilized multiplicative triplets to realize scalable privacy-preserving machine learning. Nevertheless, this method incurs a substantial communication cost.

2) *Activation Function Method*: In order to introduce nonlinearities into the neural network, Dowlin et al. [34] used the square function as an activation function to complete the CNN prediction, but it is unsuitable for deeper neural networks. In most privacy-preserving neural network schemes, the activation function is usually replaced by an approximate polynomial. Ma et al. [14] utilized the Sigmoid function as the activation function and made the function expansion by Taylor series at zero point. Subsequently, the resulting lower-degree polynomial was utilized for the computation of the activation function under ciphertext. Nevertheless, relying solely on the Taylor series approximation at zero point to approximate the Sigmoid function is not recommended. Certain approaches incorporate the derivative of the function to approximate the Relu function. Both Hesamifard et al. [17] and Kwabena et al. [24] respectively presented that utilized the approximate polynomial of the Sigmoid function to approximate the derivative of the Relu function, and then performed indefinite integration to reach the approximate polynomial of the Relu function. Their method integrates the Sigmoid function directly and approximates Taylor series at zero point at last. Although both methods are the same as the Softplus function mentioned by Chabanne et al. [19], they also face the approximation problem only around certain points. In addition, Taylor series at only zero point would not fit the entire function well, resulting in low classification accuracy in CNNs.

III. PRELIMINARY

In this section, we give a brief introduction to the relevant knowledge involved in the PCCS, which includes the CNN structure and the DT-PKC cryptosystem [27]. Furthermore, the main notations for the scheme are enumerated in Table I.

TABLE I
NOTATIONS

$\mathbb{Z}_N$	Residue class ring modulo N
Φ	convolutional kernels
W	weights in the fully-connected layer
$\mathfrak{B}$	biases in the fully-connected layer
Y	convoluted image
Y'	activated image
Y''	pooled image
$\lambda / (\lambda_1, \lambda_2)$	strong private key / partial strong private keys
$PMDec'$	partial decryption with λ_1
$PMDec''$	partial decryption with λ_2
pk_z / sk_z	the public or private key of user z
$pk_i = (N, \mathbf{g}, h_i)$	the public key for user i
$sk_i = \theta_i$	the weak private key for user i
pk_ψ	the union public key
$\ \cdot\ _{pk_\psi}$	ciphertext encrypted by pk_ψ
$SDec$	decryption with the weak private key sk_i
$MDec$	decryption with the strong private key λ

A. CNN

The CNN possesses the capability to acquire a multitude of mapping relationships without necessitating precise mathematical expressions between input and output. It is noteworthy to mention that the incorporation of convolutional, activation, and pooling layers enables the attainment of a deeper CNN. Additionally, the subsequent fully-connected layer can also be prevalent in multi-layer configurations.

The activation function bears resemblance to the human neuron as it executes nonlinear transformations on the input signal. Common activation functions encompass the Sigmoid function, the Tanh function, the Relu function, among others. By implementing the activation function on each neuron within the preceding layer, the input for the neuron in the present layer is derived. Typically, the Softmax function is employed in the output layer to conduct Softmax processing on the classification vector, thereby yielding the associated probability.

B. DT-PKC

The DT-PKC cryptosystem, proposed by Liu et al. [27], is built upon the Paillier [44] and BCP [36] cryptosystems and serves as a suitable toolkit for multiple keys. In this paper, we denote the two partial decryption process as $PMDec'$ and $PMDec''$ with the partial strong private keys λ_1, λ_2 divided by the strong private key λ in DT-PKC. Only when both parts are completed, the ciphertext can be fully decrypted into plaintext. The detail of the scheme is as follows.

First, given two large prime numbers p, q , pick $\theta_i \in [1, N/4]$ uniformly at random, choose a generator $\mathbf{g}$, and then compute $h_i = \mathbf{g}^{\theta_i} \bmod N^2$, where $N = pq$. Define an equation $L(x) = (x - 1)/N$. The public key pk_i for user i is $pk_i = (N, \mathbf{g}, h_i)$, and the weak private key sk_i for user i is $sk_i = \theta_i$. Assume that the ciphertext of the plaintext $m \in \mathbb{Z}_N$ can be denoted as $\|m\|_{pk_i} = (A, B)$, where $A = h_i^r (1 + mN) \bmod N^2$, $B = \mathbf{g}^r \bmod N^2$ with uniformly at random $r \in [1, N/4]$. The partial decryption process is:

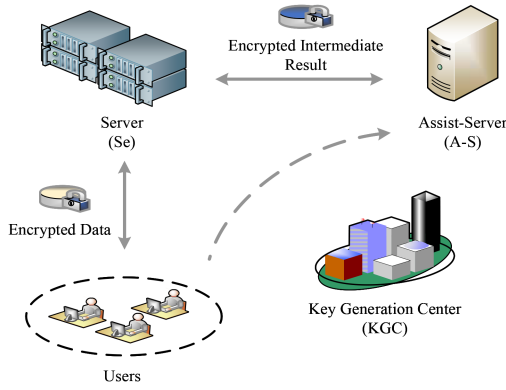


Fig. 1. System framework of PCCS.

$$PMDec' : ST_1 = (A)^{\lambda_1} = g^{r\theta_i\lambda_1}(1 + mN\lambda_1) \bmod N^2,$$

$$PMDec'' : ST_2 = (A)^{\lambda_2} = g^{r\theta_i\lambda_2}(1 + mN\lambda_2) \bmod N^2.$$

The plaintext m can be obtained as follows:

$$\begin{aligned} m &= L(ST_1 \cdot ST_2) \\ &= L(g^{r\theta_i\lambda_1}(1 + mN\lambda_1) \bmod N^2 \\ &\quad \cdot g^{r\theta_i\lambda_2}(1 + mN\lambda_2) \bmod N^2), \end{aligned} \quad (1)$$

where $\lambda_1 + \lambda_2 \equiv 0 \bmod \lambda$ and $\lambda_1 + \lambda_2 \equiv 1 \bmod N^2$.

In particular, given two ciphertexts $\|m_1\|_{pk_i}$ and $\|m_2\|_{pk_i}$ encrypted with the public key pk_i , they satisfy the following homomorphic properties:

- 1) $\|m_1\|_{pk_i} \cdot \|m_2\|_{pk_i} = \|m_1 + m_2\|_{pk_i}$.
- 2) $\|\eta m\|_{pk_i} = \|m\|_{pk_i}^\eta$, where $\eta \in \mathbb{N}$.

IV. SYSTEM OVERVIEW

In this section, we introduce the system model and security requirements.

A. System Model

Our scheme includes four entities: Key Generation Center (KGC), users, Server (Se), and Assist-Server (A-S). User data is encrypted and sent to Se, where the classification process takes place through the interaction between Se and A-S. The classification results are then returned to the users in ciphertext form. The system framework is depicted in Fig. 1.

- 1) *KGC*: KGC is responsible for the distribution and management of all the public and private keys in system. It is a trusted independent entity.
- 2) *Users*: Users securely contribute their local data, encrypted using their respective public keys, to Se for classification within the PCCS. Importantly, the private key of the user remains protected and is not exposed.
- 3) *Se*: Se is responsible for both data collection from users and the utilization of a fully-trained classification model. It collaborates with A-S to delegate partial computing tasks for classification. Notably, Se possesses exclusive knowledge of the well-trained model's parameters, as well

as the partial strong private key $MK_1 = \lambda_1$ and the ability to compute users' union public key pk_ψ .

- 4) *A-S*: A-S is responsible for computing a portion of the relevant ciphertext and generating the approximate polynomial table required during the activation process in PCCS, which is convenient for Se to choose and calculate. In particular, it can partially decrypt the ciphertext with another partial strong private key $MK_2 = \lambda_2$.

B. Threat Model

In PCCS, we assume that both Se and A-S are curious-but-honest, that is, Se and A-S strictly adhere to the various protocols established in PCCS, while also maintaining an interest in collecting and acquiring private information during the classification process (A-S should also be curious about the model parameters).

Assuming that KGC is a trusted entity and there is no collusion between Se and A-S. Based on the above premise, we introduce an adversary $\mathcal{A}$ aiming at obtaining the original user data, the intermediate data, the model parameters, and the predicted results. Specifically, it is not possible for $\mathcal{A}$ to corrupt both Se and A-S simultaneously, nor can it corrupt all users concurrently.

- 1) $\mathcal{A}$ could eavesdrop on the transmitted information during all interactions.
- 2) $\mathcal{A}$ could corrupt Se or A-S to guess all ciphertexts during the classification process with partial decryption access.
- 3) $\mathcal{A}$ could corrupt one or more users to deduce the ciphertexts from other users.

C. Design Goals

Based on the threat model, our design goals are as follows.

- 1) *Data privacy and model privacy*: Users' raw data, intermediate results, model parameters, and classification results need to be protected.
- 2) *Classification accuracy*: Different approximation selection and computation methods of the activation function will bring different accuracy, so we need to ensure classification accuracy in the privacy-preserving scheme.
- 3) *Lightweight users*: Users are expected to upload their data and accept the results without the need for frequent interactions or constant online presence.
- 4) *Multiple users*: A single-key scheme cannot fulfill the practical requirement for each participant to possess his private key. Therefore, the scheme considers the scenario of multiple keys.

V. CONSTRUCTION OF PCCS

In this section, to realize PCCS, we propose three indispensable security protocols that have to be utilized in the PCCS: SMP, SAC, and APP. Then, we elaborate on the specific details of the PCCS based on DT-PKC.

A. Secure Multiplication Protocol

When classifying on a CNN, both the convolutional layer and the fully-connected layer necessitate multiplication operations.

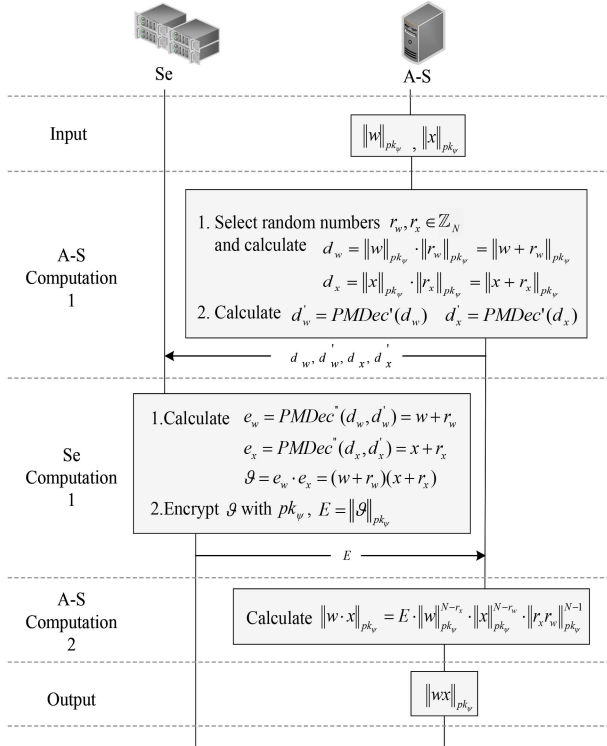


Fig. 2. Secure multiplication protocol.

To address this requirement, the SMP can be employed, as depicted in Fig. 2.

Assume that A-S receives $\|w\|_{pk_\psi}$ and $\|x\|_{pk_\psi}$ encrypted by the union public key pk_ψ . Since A-S only has partial decryption access, it will have to interact and cooperate with the Se that also has partial access to obtain the ciphertext $\|wx\|_{pk_\psi}$ corresponding to plaintext wx .

B. Secure Activation Function Computing Protocol

This study introduces the approximation of the activation function through the expansion of the Taylor series at the piecewise points within the SAC protocol. Specifically, the activation function expands the Taylor series at each integer point, encompassing all integer points within the data input. Moreover, this paper utilizes polynomials that operate within specific ranges, such as three polynomials $f_0(x)$, $f_1(x)$ and $f_2(x)$ are generated at zero, one and two points, their corresponding intervals are $(-0.5, 0.5]$, $(0.5, 1.5]$ and $(1.5, 2.5]$. To realize the SAC, we design the following three aspects.

1) *Tanhplus Function:* Inspired by the Softplus function, we construct the Tanhplus function that is also bounded, continuous and differentiable. We first design the TanhTrans function:

$$\text{TanhTrans} = \frac{1}{2} \frac{e^x - e^{-x}}{e^x + e^{-x}} + \frac{1}{2}. \quad (2)$$

Utilize TanhTrans to approximate the derivative of the Relu function, then do indefinite integral to the TanhTrans function

Table T			Table T'	
Index	Approximate polynomials		Index	Approximate polynomials
...	...		...	...
0	$f_0(x) = a_0^0 + a_1^0 x + a_2^0 x^2 + a_3^0 x^3$	$r_3(\text{index} + r_2) \Rightarrow \text{index}$	$r_3(0 + r_2)$	$f_0(x) = a_0^0 + a_1^0 x + a_2^0 x^2 + a_3^0 x^3$
1	$f_1(x) = a_0^1 + a_1^1 x + a_2^1 x^2 + a_3^1 x^3$		$r_3(1 + r_2)$	$f_1(x) = a_0^1 + a_1^1 x + a_2^1 x^2 + a_3^1 x^3$
2	$f_2(x) = a_0^2 + a_1^2 x + a_2^2 x^2 + a_3^2 x^3$		$r_3(2 + r_2)$	$f_2(x) = a_0^2 + a_1^2 x + a_2^2 x^2 + a_3^2 x^3$
...	...		...	...
i	$f_i(x) = a_0^i + a_1^i x + a_2^i x^2 + a_3^i x^3$		$r_3(i + r_2)$	$f_i(x) = a_0^i + a_1^i x + a_2^i x^2 + a_3^i x^3$
...	...		...	...

Fig. 3. Approximate Polynomials.

to obtain the Tanhplus function:

$$\text{Tanhplus} = \frac{1}{2} (\ln(e^x + e^{-x}) + x). \quad (3)$$

At last, the Relu function is replaced with the Tanhplus function. We are again motivated to suggest that the activation function employs the Tanhplus function to extend a four-series Taylor expansion (a trade-off between accuracy and efficiency, for further details see Section VII) at each integer piecewise point. The resulting approximate polynomials are then substituted for the activation function in the SAC protocol.

2) *SAC:* This subsection describes the specific details of the SAC protocol shown in Fig. 4. In SAC, A-S has ciphertext $\|x\|_{pk_\psi}$ and calculates activation function through interaction with Se. We mention that the activation function takes the form of an expanded Taylor series at piecewise points, which can effectively approximate the function itself. However, for a certain input $\|x\|_{pk_\psi}$, in order to get its corresponding activation polynomial, the ciphertext must be decrypted in the process, which will lead to data privacy leakage.

To solve the problem of privacy leakage, we let the activation function expand 4-series Taylor expansion at piecewise points on A-S according to the plaintext range, and generate an initial table T containing relevant approximate polynomials. Based on T, A-S then performs related homomorphic operations on $\|x\|_{pk_\psi}$, and at the same time performs corresponding operations on the index from T to generate the table T', as shown in Fig. 3. At this time, Se has $\{\|x\|_{pk_\psi}, \tilde{v}, T'\}$ and it will look for new index in T' according to index $\tilde{v}$ in the process of Se computation 1 of Fig. 4, then $f_{v_N}(x)$ applied to $\|x\|_{pk_\psi}$ could be selected.

In this case, there is no need for $\|x\|_{pk_\psi}$ to be decrypted both on two servers. The information exposed to Se solely consists of the intermediate outcome with the inclusion of random noise. On the other hand, A-S assumes responsibility for managing the table and lacks knowledge regarding the specific activation polynomial chosen by Se, which ensures data privacy without regard for plaintext based on a specific index. During the interaction, $\{\|x^2\|_{pk_\psi}, \|x^3\|_{pk_\psi}\}$ would also be generated in application to the activation polynomial calculations.

3) *Homogenization Algorithm:* It is acknowledged that the DT-PKC can be employed for integers. Nevertheless, the weight w and bias b of fully-trained models in CNN typically exist in decimal format. Consequently, users often preprocess the

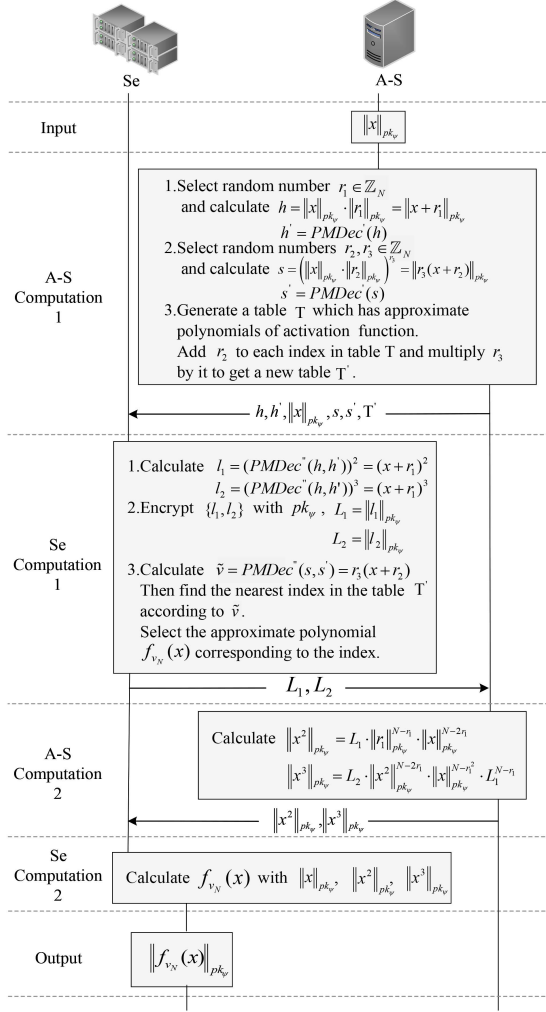


Fig. 4. Secure activation function computing protocol.

contributed image pixels, converting them into integer representation prior to uploading them to the server. During the process of encryption, the data undergo expansion by multiples to yield integers, which are subsequently encrypted using public keys, that is, x and w respectively in $\|x\|_{pk_\psi}$ and $\|w\|_{pk_\psi}$ are actually expanded integer data. It is noteworthy to mention that the variable x in the approximate polynomial $f_i(x)$ represents an unexpanded integer data. Furthermore, it is important to acknowledge that the powers of the independent variable differ for each term within the polynomial.

In the final step of SAC, $\|f_{v_N}(x)\|_{pk_\psi}$ is calculated according to $\{\|x\|_{pk_\psi}, \|x^2\|_{pk_\psi}, \|x^3\|_{pk_\psi}\}$. If $x = y \cdot 10^\beta$, where x is an integer corresponding to the decimal y , then the input $f_{v_N}(x)$ here is actually $f_{v_N}(y)$ and $f_{v_N}(x)$ in $\|f_{v_N}(x)\|_{pk_\psi}$ is indeed expressing the expanded integer form. As the result of the $f_{v_N}(y)$ calculation may be a decimal unsupported homomorphic encryption, therefore it can be directly unavailable. Apart from that, Ma et al. [14] substituted $\{\|x^2\|_{pk_\psi}, \|x^3\|_{pk_\psi}\}$ into calculation without any operations, which may result in incorrect activation output under ciphertext. We know $x = y \cdot 10^\beta$, then $x^2 = y^2 \cdot 10^{2\beta}$ and $x^3 = y^3 \cdot 10^{3\beta}$, namely, different expansion coefficients can result in inaccurate activation calculation.

Algorithm 1: Homogenization Algorithm.

Input: $\{\|x\|_{pk_\psi}, \|x^2\|_{pk_\psi}, \|x^3\|_{pk_\psi}, f_{v_N}(x)\}$

Output: $\|f_{v_N}(x)\|_{pk_\psi}$

1: For polynomial $f_{v_N}(x)$, let $y = x$, we can get:

$$f_{v_N}(y) = a_0^{v_N} + a_1^{v_N}y + a_2^{v_N}y^2 + a_3^{v_N}y^3.$$

2: Suppose that by expanding the multiple 10^α , the coefficients of the polynomial $f_{v_N}(y)$ just expand to an integer. We define $[a_i^{v_N}]$ to mean that $a_i^{v_N}$ expands by a factor 10^α , then multiply both sides of the formula by 10^α :

$$10^\alpha \cdot f_{v_N}(y) = [a_0^{v_N}] + [a_1^{v_N}]y + [a_2^{v_N}]y^2 + [a_3^{v_N}]y^3,$$

multiply both sides by $10^{3\beta}$ and let $x = 10^\beta y$:

$$\begin{aligned} & 10^{3\beta} \cdot 10^\alpha \cdot f_{v_N}(y) \\ &= [a_0^{v_N}] \cdot 10^{3\beta} + [a_1^{v_N}] \cdot 10^{2\beta} \cdot (10^\beta y) \\ &+ [a_2^{v_N}] \cdot 10^\beta (10^\beta y)^2 + [a_3^{v_N}] \cdot (10^\beta y)^3 \\ &= [a_0^{v_N}] \cdot 10^{3\beta} + [a_1^{v_N}] \cdot 10^{2\beta} x + [a_2^{v_N}] 10^\beta x^2 \\ &+ [a_3^{v_N}] x^3. \end{aligned}$$

3: After $f_{v_N}(y)$ has been preprocessed, the right side of the equation ensures that the coefficients and independent variables are integer form, facilitating homomorphic encryption. Based on the homomorphic property, we get:

$$\begin{aligned} & \left\| 10^{3\beta} \cdot 10^\alpha \cdot f_{v_N}(y) \right\|_{pk_\psi} \\ &= \left\| [a_0^{v_N}] \cdot 10^{3\beta} \right\|_{pk_\psi} \cdot \left\| x \right\|_{pk_\psi}^{[a_1^{v_N}] \cdot 10^{2\beta}} \\ & \cdot \left\| x^2 \right\|_{pk_\psi}^{[a_2^{v_N}] \cdot 10^\beta} \cdot \left\| x^3 \right\|_{pk_\psi}^{[a_3^{v_N}]} \end{aligned}$$

We denote $f_{v_N}(x) = 10^{3\beta} \cdot 10^\alpha \cdot f_{v_N}(y)$, and it will obtain $\|f_{v_N}(x)\|_{pk_\psi}$.

4: Correctness:

$$\begin{aligned} & \text{Dec} \left(\left\| 10^{3\beta} \cdot 10^\alpha \cdot f_{v_N}(y) \right\|_{pk_\psi} \right) \\ &= 10^{3\beta+\alpha} \cdot f_{v_N}(y) = f_{v_N}(x). \end{aligned}$$

5: **return** $\|f_{v_N}(x)\|_{pk_\psi}$.

In order to address the aforementioned issues, Algorithm 1 introduces the HA as a proposed solution for solving the calculation in the final step of the SAC protocol.

C. Average Pooling Protocol

The purpose of the APP protocol is to perform calculations on the input ciphertext and, through interaction, ultimately achieve an average over the plaintext using the DT-PKC. At the pooling layer, the APP protocol depicted in Fig. 5 is employed to average ciphertexts within feature maps.

D. Details of PCCS

The PCCS scheme is established on the premise of classifying CNN with privacy protection in a multi-key environment. In practice, the CNN framework necessitates adjustments to accommodate specific requirements, wherein the convolutional layer, activation layer, pooling layer, and fully-connected

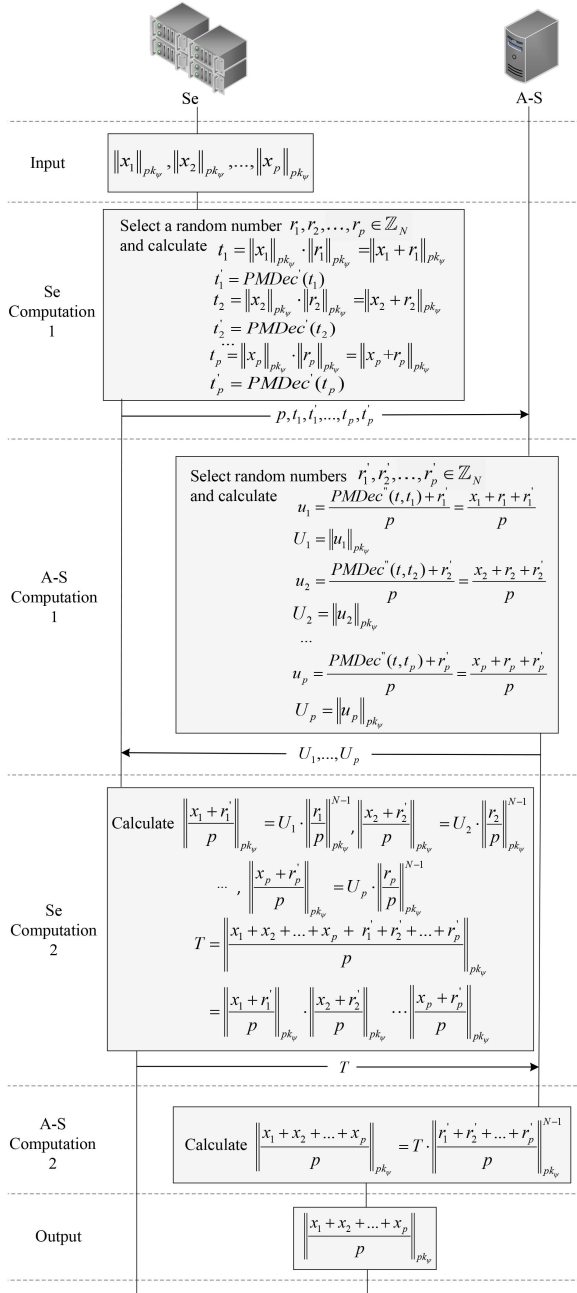


Fig. 5. Average pooling protocol.

layer serve as indispensable components. Consequently, we opt for the most conventional and uncomplicated CNN framework to explicate the application of our PCCS scheme in classification.

On the one hand, we assume that the type and size of the data contributed by each user are consistent with the size $n \cdot n$. On the other hand, both the activation layer and the fully-connected layer adopt the Tanhplus function, the number of the convolutional kernel is noted as s , the stride is set to one, and the pooling area is $\sqrt{p} \cdot \sqrt{p}$.

In PCCS, the trusted KGC generates and allocates public and private key pair $\{pk_z, sk_z\}, z \in [1, b]$ to each user and allocates Se and A-S partial strong private keys λ_1, λ_2 , where

b is the number of users. Then users upload data $\|X^z\|_{pk_z}, z \in [1, b], X \in \mathbb{Z}_N$ and random numbers $\|R^z\|_{pk_z}, z \in [1, b], R \in \mathbb{Z}_N$, encrypted by their respective public keys. Se has well-trained models $\Phi^k, k \in [1, s], W^\gamma, \gamma \in \{1, 2\}$, and $\mathfrak{B}^\mu, \mu \in \{1, 2\}$, where Φ are the convolutional kernels, W are the weights in the fully-connected layer, $\mathfrak{B}$ are the biases in the fully-connected layer, s is the number of convolutional kernels, and γ and μ are the layer number of the fully-connected layer. Se then collects data to complete classification with the assistance of computational resources from A-S. We list the process of PCCS in Fig. 6 and give the detailed illustration as follows.

Step-1(@Se): Se collects $\|X^z\|_{pk_z}$ and $\|R^z\|_{pk_z}$, while encrypting the model parameters with the union public key pk_{ψ} to get $\|\Phi^k\|_{pk_{\psi}}, \|W^\gamma\|_{pk_{\psi}}, \|\mathfrak{B}^\mu\|_{pk_{\psi}}$. Then Se sends all of these to the A-S.

Step-2(@A-S): Receiving $\|X^z\|_{pk_z}, \|R^z\|_{pk_z}, \|\Phi^k\|_{pk_{\psi}}, \|W^\gamma\|_{pk_{\psi}}, \|\mathfrak{B}^\mu\|_{pk_{\psi}}$, A-S calculates $h_{\psi', z}$ corresponding to each user's public key and uses $h_{\psi', z}$ to complete $\|X^z\|_{pk_{\psi}}, \|R^z\|_{pk_{\psi}}$ with the union public key pk_{ψ} .

After calculating $\|X^z\|_{pk_{\psi}}, \|R^z\|_{pk_{\psi}}$, it is possible to proceed with the CNN layers.

Convolutional Layer: For user z , $\|x_{ij}^z\|_{pk_{\psi}} \in \|X^z\|_{pk_{\psi}}$ denotes ciphertext representing i -th row and j -th column of user data x , where $i, j \in [1, n]$. $\|\phi_\sigma^k\|_{pk_{\psi}} \in \|\Phi^k\|_{pk_{\psi}}$ denotes σ -th data of k -th convolutional kernel, where $\sigma \in [1, t^2]$ and t^2 is the size of the convolutional kernel. $Y^{z, k}$ is recorded as the k -th layer map from the product of encrypted data and the k -th convolutional kernel, and each $y_{u,v}^{z, k} \in Y^{z, k}$ denotes the u -th row and v -th column result of k -th layer map, where $u, v \in [1, n + 1 - t]$. Assuming that count variables are $\sigma = 1, res = 0$, we can obtain $y_{u,v}^{z, k}$ denoting the convolutional result from k -th map of user z .

Activation Layer: Since $y_{u,v}^{z, k} \in Y^{z, k}$, where $u, v \in [1, n + 1 - t]$, denotes data in the k -th layer u -th row v -th column on feature map formed after previous layer data passes through the activation layer.

The expansion of the activation function at each piecewise point is dependent on the size of the data and can be stored in the table T for reuse in A-S. Furthermore, the values of α and β in the homogenization operation of HA are expanded multiples that ensure all approximate polynomial coefficients and inputs are integers. As the output of the SAC protocol is in Se, it is equivalent to the transmission of $Y^{z, k}$ from A-S to Se and subsequent progression to the next layer.

Step-3(@Se): Given pooling area $\sqrt{p} \cdot \sqrt{p}$, we note that k -th layer map after pooling is $Y''^{z, k}$, so $y_{u'', v''}^{z, k} \in Y''^{z, k}$ denotes data in k -th layer u'' -th row v'' -th column, where $u'', v'' \in [1, n + 1 - t/\sqrt{p}]$.

Pooling Layer: After average pooling for received data, it will get a new map $Y''^{z, k}$ by Step-3(@Se) utilizing APP.

Convolutional-Activation-Poolingⁿ: Convolutional, activation, and pooling operations are typically performed based on specific requirements, with n denoting the number of operations involved in the intermediate process. While the parameters of these convolutional, activation, and pooling layers vary, the practical calculations align with the aforementioned methods, thus avoiding repetition.

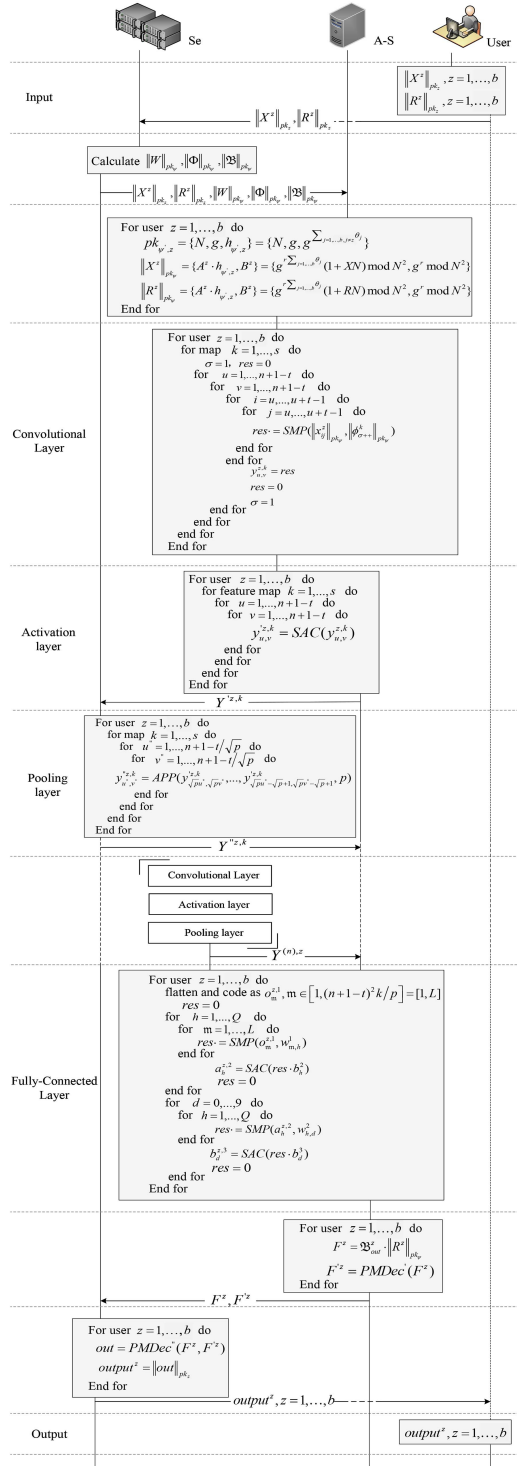


Fig. 6. Details of PCCS.

Step-4(@A-S): Note that output mapping of previous layer is $Y^{n,z}$, along with previously received model parameters $\|W^{\gamma}\|_{pk_{\psi}}, \|\mathcal{B}^{\mu}\|_{pk_{\psi}}$, A-S continues with the fully-connected layer.

Fully-connected Layer: $\|w^{\gamma}\|_{pk_{\psi}} \in \|W^{\gamma}\|_{pk_{\psi}}$ denotes weight from γ -th fully-connected layer, while $\|b^{\mu}\|_{pk_{\psi}} \in \|\mathcal{B}^{\mu}\|_{pk_{\psi}}$ denotes bias from μ -th fully-connected layer. As

the fully-connected layer is a classical multi-layer perceptron model, the received $Y^{n,z}$ needs to undergo flatten operation to elongate data from matrix into a one-dimensional vector $O_m^{z,1} \in O^z$, $m \in [1, (n+1-t)^2 k/p] = [1, L]$ as the m -th input data of the first layer, $a_h^{z,2} \in \mathcal{A}^z$, $h \in [1, Q]$ as the h -th input data of the second layer and $b_d^{z,3} \in \mathcal{B}_{out}^z$, $d \in [0, 9]$ as the d -th output of the third layer, where L and Q are the number of the neurons for the corresponding layers. It is worth noting that some experiments show that the presence or absence of hidden layers in the fully-connected layer has minimal impact on classification. Consequently, we opt for a single hidden layer feedforward network as the fully-connected layer, consisting of three layers of neurons $L, Q, 10$.

Upon completion of the fully-connected layer, we employ a likelihood function to calculate the likelihood probabilities of each category and subsequently identify the classification result as the number with the highest probability. However, in the classification phase, there will be no back propagation of the gradient, so the output maximum can be availed to predict the classification results. By the output $\mathcal{B}_{out}^z$ from the fully-connected layer, we calculate F^z, F^{z*} and send them to Se.

Step-5(@Se): Se decrypts the received data using the partial strong private key, and then generates the result encrypted using the user's public key pk_z and sends it to user z .

Upon receiving the ciphertext output $output^z$, user z decrypts it using the private key sk_z and eliminates the random noise R^z in order to determine the classified result based on the maximum value.

VI. THEORETICAL ANALYSIS

In this section, we provide a theoretical analysis of PCCS in terms of security, communication cost, computational complexity, and functionality. Due to the page constraints, we put the security proof and security analysis in *Appendix*, available online.

A. Efficiency Analysis

In this section, we theoretically analyze the communication cost and computational complexity in PCCS based on the DT-PKC cryptosystem.

1) **Communication Cost:** We first conduct an examination of the communication costs associated with SMP, SAC, and APP in the context of PCCS. These costs are illustrated in the right part of the Table II. Subsequently, the analysis of communication costs within PCCS is explicated in another part of the Table II, where $|\cdot|$ denotes the binary length of ciphertext. It is important to note that our examination focuses on the communication costs for each round pertaining to the data sender. We solely consider the communication costs of a single user. However, for A-S and Se, they must account for the communication of all users.

2) **Computational Complexity:** We have analyzed the computational complexity of PCCS, which is presented in the Table III. In Table III, Enc denotes encryption with DT-PKC cryptosystem, MM stands for modular multiplication with integer, and SMP indicates relevant operations required in this protocol. Similarly, SAC and APP indicate related computation required in their protocols respectively. PMDec' represents

TABLE II
COMMUNICATION COST OF PCCS

Phase (PCCS)	Communication Cost	Phase (SMP)	Communication Cost
Input:user	$(n^2 + 10) N^2 $	A-S Computation 1	$4 N^2 $
Step 1 (Se Computation 1)	$(bn^2 + st^2 + (L+1)Q + 20) N^2 $	Se Computation 1	$ N^2 $
Step 2 (A-S Computation 1)	$bs(n+1-t)^2$	Phase (SAC)	Communication Cost
Step 3 (Se Computation 2)	$L \cdot N^2 \text{ or } (s(n+1-t)^2/p) N^2 $	A-S Computation 1	$5 N ^2 + T$
Step 4 (A-S Computation 2)	$20b N^2 $	Se Computation 1	$2 N^2 $
Step 5 (Se Computation 3)	$10b N^2 $	A-S Computation 2	$2 N^2 $
		Phase (APP)	Communication Cost
		Se Computation 1	$2p N^2 $
		A-S Computation 1	$p N^2 $
		Se Computation 2	$ N^2 $

TABLE III
COMPUTATIONAL COMPLEXITY OF PCCS

Phase	Computational Complexity	
Input	—	$(n^2 + 10)\text{Enc}$
Step 1 (Se Computation 1)	—	$bMM+(st^2 + (L+1)Q+10)\text{Enc}$
Step 2 (A-S Computation 1)	—	$(11+n^2)b-1$
	Convolutional Layer	$t^2sb(n+1-t)^2(\text{SMP} + \text{MM})$
	Activation Layer	$sb(n+1-t)^2\text{SAC}$
Step 3 (Se Computation 2)	Pooling Layer	$sb((n+1-t)^2/p)\text{APP}$
Step 4 (A-S Computation 2)	Fully-Connected Layer	$(10+Q)\text{SAC} + Q(10+L)(\text{SMP} + \text{MM})$
	—	$10b(\text{MM} + \text{PMDec}')$
Output	—	$10(\text{SDec} + \text{MS})$

computation required to partially decrypt ciphertext using a partially strong private key, SDec denotes decryption with user's private key, and MS denotes modular subtraction with integer. In the input and output phases, because it is meaningless to analyze the computational complexity of all users, we are only concerned with the computational complexity of each user.

3) *Functionality*: In this subsection, we give functionality comparisons for different neural network classification schemes. In comparison schemes, the schemes in [20], [29] are primarily employed in the context of multilayer perceptron networks, with certain investigations concentrating on CNN classification [25], [31], [34]. Concurrently, the schemes in [25], [29], [31] facilitate model training, excluding model classification. Drawing from the primary obstacles outlined in Section I, the comparative assessment centers on nine facets, as delineated in Table IV. As evidenced by the data in Table IV, the schemes presented in [17], [25], [34] comprise two entities and do not account for security against Se. Conversely, the remaining schemes encompass three entities and ensure security against Se, with the exception of [20]. All comparison schemes have the capability to ensure security for users, although certain studies fail to address security concerns regarding A-S [20], [29], [31]. In particular, the security of the private key is disregarded in studies [20] and [29] due to its shared nature between the cloud and user. Furthermore, only [31] and our proposed PCCS possess the ability to accommodate multiple keys. Upon comparison, it

- 1) *Convolution*: kernel size $5 \times 5 \times 20$, stride (1,1).
- 2) *Activation*: function Relu.
- 3) *Pooling*: average pooling area 2×2 , stride 2.
- 4) *Convolution*: kernel size $5 \times 5 \times 50$, stride (1,1).
- 5) *Activation*: function Relu.
- 6) *Pooling*: average pooling area 2×2 , stride 2.
- 7) *Fully-connection*: input nodes 800, hidden nodes 500, output nodes 10.

Fig. 7. CNN construction (MNIST dataset).

becomes evident that our PCCS not only safeguards the privacy of data and models for three parties (Se, A-S, and users), but also offers lightweight functionality and supports multiple users.

In order to facilitate a comprehensive comparison of communication costs and computation overheads among users, we have presented the costs associated with these schemes in Table IV, where $q, t \in \mathbb{Z}_q, 0 \leq t \leq q, \kappa$ is the number of classification outputs, θ is the number of neurons in the input layer, $|B|$ denotes the mini-batch size, and d represents the number of rounds. Moreover, the symbols Trans, Dot, Exp, Div, and Mul_{matrix} have been employed to respectively denote the transposition operation, dot multiplication, modular exponentiation, division, and matrix multiplication. Upon examination of Table IV, it becomes evident that our communication costs exhibit a relatively low magnitude.

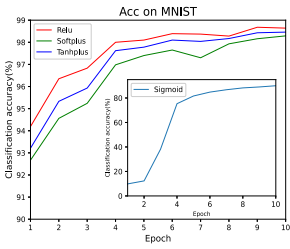
VII. PERFORMANCE EVALUATION

In this section, a series of experiments were conducted to assess the performance of the PCCS scheme in terms of classification accuracy and efficiency. The experiments were executed on two machines: one equipped with an Intel(R) Xeon(R) Silver 4110 sixteen-core CPU @ 2.10 GHz and V100-PCIE-16 GB GPU with 64 GB RAM running on the Windows 10 Operating System (Se) for the fully-trained model, and another machine with an Intel core i7-9750H six-core CPU @ 2.6 GHz and 8 GB RAM running on the Linux Operating System (A-S) for the classification task. The cryptosystem utilizes the DT-PKC based on python.

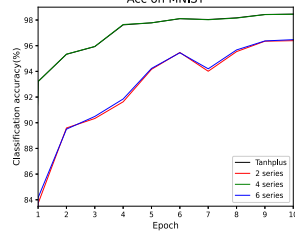
Initially, we select the classical MNIST dataset, comprising 60,000 training handwritten digits and 10,000 test digits from "0" to "9". Each digit encompasses 784 features, representing a 28×28 pixel image. Subsequently, we utilize the CIFAR-10 color image dataset, which closely resembles universal objects, encompassing a total of 10 categories of RGB color images. Each image has dimensions of 32×32 , and each category contains 6,000 images. The dataset comprises a total of 50,000 training images and 10,000 test images. Using the deep learning framework of Pytorch 1.2.0, we have developed a CNN model for the MNIST dataset. The detailed architecture of this model is illustrated in Fig. 7. However, due to the increased complexity of the images in the CIFAR-10 dataset, we have opted to employ the classical VGG 16 model.

TABLE IV
FUNCTIONALITY COMPARISON

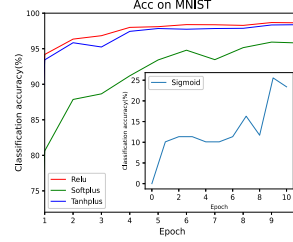
Scheme	Cryptographic techniques	Security against Se	Security against users	Security against A-S	Number of entities	Communication cost for users	Computation overhead for users	Support Multiple keys	Cloud and user share private key
[17]	LHE	-	Yes	Yes	Two	$(n^2 + 10) q $	$n^2(\text{Trans} + \text{MM}) + 10\text{Dot}$	No	No
[20]	Paillier	No	Yes	No	Three	$(n\theta + \kappa) N^2 $	$(2n\theta + 2\kappa)\text{Exp} + n\theta\text{MM} + \kappa\text{Div}$	No	Yes
[25]	YASHE'	-	Yes	Yes	Two	$dn^2 q + d\kappa \tau $	$(2n^2 + 2\kappa)\text{MM}$	No	No
[29]	Paillier	Yes	Yes	No	Three	$(n\theta + \kappa) N^2 $	$(2n\theta + 2\kappa)\text{Exp} + n\theta\text{MM} + \kappa\text{Div}$	No	Yes
[31]	SMC	Yes	Yes	No	Three	$n\theta + (B + \theta)d$	12Mul_{matrix}	Yes	No
[34]	YASHE'	-	Yes	Yes	Two	$n^2 q + \kappa \tau $	$(2n^2 + 2\kappa)\text{MM}$	No	No
PCCS	DT-PKC	Yes	Yes	Yes	Three	$(n^2 + 10) N^2 $	$2(n^2 + 10)\text{Exp} + (2n^2 + 30)\text{MM} + 20\text{Div}$	Yes	No



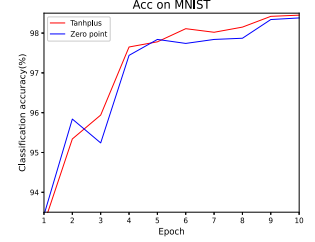
(a) Different activation functions comparison



(b) Tanhplus with 2,4,6 Taylor series at piecewise points



(c) Different activation functions at piecewise points



(d) Tanhplus at piecewise points and zero point

Fig. 8. Classification accuracy.

A. Classification Accuracy

To evaluate the performance of our constructed CNNs, we have conducted 10 epochs of training on various architectures using the MNIST training data. Subsequently, we have assessed the classification accuracy of these CNNs on the test dataset, resulting in different accuracy rates for each case, as presented in Fig. 8.

Fig. 8(a) illustrates the impact of various activation functions (Relu, Tanhplus, Softplus, and Sigmoid) on the classification accuracy of the MNIST dataset. To obtain the classification accuracy, we conducted 10 epochs of training on a non-privacy-protected CNN. The results depicted in Fig. 8(a) indicate that the Sigmoid function exhibits significantly lower classification accuracy compared to the other three activation functions. The corresponding graph of the Sigmoid function is presented in the lower right corner. The Tanhplus function, which is continuous and derivable, exhibits a higher classification accuracy compared to the Softplus function. Although the Relu function also demonstrates high accuracy, it is continuous but non-derivative at the zero point.

In Fig. 8(b), the impact of the Tanhplus function on classification accuracy is illustrated using different Taylor series (2, 4, and 6) at each integer piecewise point in the MNIST dataset. Initially, we expand the Taylor series for the Tanhplus function at each integer point using series 2, 4, and 6, and then employ the corresponding Taylor expansion as an activation function. Subsequently, a series of 10-epoch experiments were conducted on the constructed CNN using the aforementioned activation

functions, resulting in varying classification accuracies. It is evident that the classification accuracy of the 2-series and 6-series is relatively similar, whereas the 4-series exhibits nearly identical performance to Tanhplus and surpasses the previous two in terms of accuracy.

Fig. 8(c) illustrates the impact of different activation functions on classification accuracy, specifically focusing on the 4-series Taylor expansion at each piecewise integer point within the MNIST dataset. The Relu function was employed as a benchmark for comparison. The findings suggest that the Tanhplus function exhibits superior performance compared to other functions in terms of accuracy when expanding the 4-series Taylor expansion at piecewise points. Fig. 8(d) illustrates the variation in classification accuracy influenced by Tanhplus with Taylor series at each piecewise point and at the zero point, using the MNIST dataset. It is evident that the accuracy of Tanhplus with Taylor series at piecewise points surpasses that at the zero point.

B. Classification Efficiency

To assess the effectiveness of the PCCS scheme, we conducted an evaluation of the PCCS scheme using the MNIST testing dataset, employing the DT-PKC cryptosystem. Our evaluation focused on various aspects, including the number of data, users, and Taylor series, to determine the running time of each user (User time), convolutional layer I (Conv 1), activation layer I (Act 2), pooling layer I (Pool 1), convolutional layer II (Conv 2), activation layer II (Act 2), pooling layer II (Pool 2), and

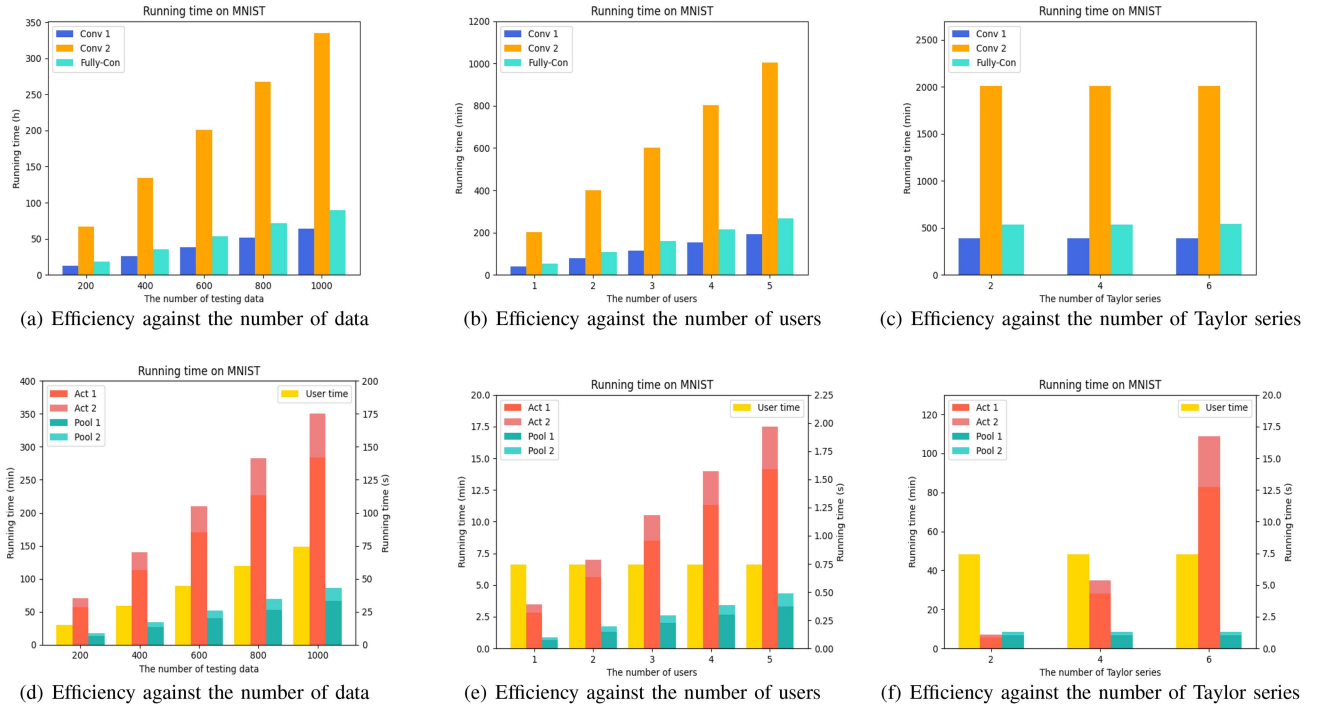


Fig. 9. Classification efficiency.

fully-connected layer (Fully-con) as depicted in Fig. 9. It is worth noting that the average encryption and decryption times for users when processing a MNIST image are very small (0.073 s and 0.001 s).

In Fig. 9(a) and (d), the efficiency of our scheme is assessed in relation to the number of testing data. The MNIST dataset is divided into 200 pieces, and the focus is on observing the runtime variation as the testing data increases. It is evident that the runtime for the user is minimal, with experimental findings indicating an average encryption and decryption time of 0.074 s for an MNIST image. This aligns with the objective of PCCS to cater to lightweight users. Among the different metrics, Conv 2 exhibits the second highest runtime, surpassed only by Fully-Con, due to its utilization of $5*5*50$ convolutional kernels on $12*12*20$ feature maps. Fully-Con exhibits the longest runtime due to its extensive execution of inner product operations, which bear resemblance to convolutional operations. Generally, as the quantity of testing data grows, the runtime of the eight metrics also increases, as these metrics are integral to the execution of image classification.

In Fig. 9(b) and (e), we assess the efficacy of our approach in relation to the number of users, with each user contributing 100 data points encrypted with their respective public keys. In this analysis, we solely focus on the individual runtime of each user. It is evident that the augmentation of metrics, except for User time, occurs as the number of users increases. This can be attributed to the collection of data from each user for image classification on both Se and A-S, leading to an increase in the running time of all metrics. This observation aligns with the PCCS scheme's ability to facilitate client-side lightweight operations.

In Fig. 9(c) and (f), we conducted an assessment of PCCS in relation to the number of Taylor series employed. Specifically,

we employed Tanhplus on a set of 100 MNIST images, utilizing 2-series, 4-series, and 6-series Taylor expansions at piecewise points for classification purposes. Our findings indicate that the computational time associated with the activation function for the 4-series is significantly lower than that of the 6-series, albeit slightly higher than that of the 2-series. With the exception of Act 1, Act 2, and Fully-con, the remaining metrics exhibit minimal variation and maintain a consistent level of performance. Based on the findings of the previous experiment, it is evident that the utilization of a 4-series Taylor expansion at piecewise points yields unparalleled accuracy compared to the other two series. Consequently, it continues to adhere to the principle of PCCS, suggesting that employing the 4-series Taylor expansion strikes a favorable balance between efficiency and accuracy.

To further assess our performance, we conducted tests and evaluations on the classification efficiency, considering the number of data, users, and Taylor series, utilizing the CIFAR-10 dataset. The outcomes pertaining to the running time are presented in Tables V–XI. Their units are consistent with the corresponding units in the experimental plots of the MNIST dataset.

Table V presents the impact of various activation functions on the classification accuracy of the CIFAR-10 dataset. The activation functions include Relu, Sigmoid, Tanhplus, and Softplus. These activation functions were individually applied to the VGG 16 CNN model, which is more intricate and non-privacy-preserving. Subsequently, the model underwent 200 epochs to determine the classification accuracy associated with each activation function.

It is evident that the Sigmoid function is unsuitable for the complex CNN network when applied to the color image CIFAR-10 dataset. Our proposed continuous and derivable Tanhplus function, serving as the activation function, exhibits

TABLE V
ACTIVATION FUNCTION FOR CIFAR-10

Activation function	25	50	75	100	125	150	175	200
Relu_CIFAR-10	82.88	83.96	86.15	87.03	87.98	91.6	93.53	93.84
Sigmoid_CIFAR-10	10	10	10	10	10	10	10	10
Softplus_CIFAR-10	32.89	24.12	38.29	66.88	63.21	74.6	85.28	90.53
Tanhplus_CIFAR-10	65.85	68.56	73.18	76.45	77.09	81.33	90.07	91.81

TABLE VI
EFFICIENCY AGAINST DATA NUMBERS FOR CIFAR-10

Data number	200	400	600	800	1000
Conv 1	13.87	27.74	41.62	55.49	69.37
Conv 2	72.36	144.72	217.08	289.44	361.79
Fully-Con	19.35	38.70	58.05	77.41	96.76

TABLE VII
EFFICIENCY AGAINST DATA NUMBERS FOR CIFAR-10

Data number	200	400	600	800	1000
user_time	16.08	32.15	48.23	64.31	80.38
Act 1	61.25	122.51	183.76	245.02	306.27
Act 2	14.40	28.80	43.20	60.84	72.00
Pool 1	14.40	28.80	43.20	60.84	72.00
Pool 2	4.25	8.50	12.76	17.01	21.26

TABLE VIII
EFFICIENCY AGAINST USER NUMBERS FOR CIFAR-10

User number	1	2	3	4	5
Conv 1	50.29	100.57	150.86	201.14	251.43
Conv 2	262.27	524.54	786.81	1049.08	1311.35
Fully-Con	70.14	140.28	210.42	280.56	350.69

TABLE IX
EFFICIENCY AGAINST USER NUMBERS FOR CIFAR-10

User number	1	2	3	4	5
Act 1	3.696	7.393	11.089	14.785	18.482
Act 2	0.871	1.741	2.612	3.483	4.354
Pool 1	0.871	1.741	2.612	3.483	4.354
Pool 2	0.257	0.514	0.771	1.017	1.284

TABLE X
EFFICIENCY AGAINST TAYLOR SERIES NUMBERS FOR CIFAR-10

Taylor series number	2	4	6
Conv 1	419.05	419.05	419.05
Conv 2	2185.58	2185.58	2185.58
Fully-Con	583.62	584.49	589.71

a higher classification accuracy compared to the Softplus function. However, it falls slightly behind the continuous and zero-point non-derivable Relu function, which remains the most accurate. Furthermore, the classification accuracy of the Tanhplus function stabilizes after 180 epochs of training. Additionally, the Softplus function displays pronounced oscillations when utilized as an activation function.

According to multiple experiments and evaluations, the graphical representations and tabular data demonstrate that the PCCS scheme exhibits a commendable classification efficiency, while also ensuring privacy protection, high accuracy, and accommodating lightweight and multiple users in image

TABLE XI
EFFICIENCY AGAINST TAYLOR SERIES NUMBERS FOR CIFAR-10

Taylor series number	2	4	6
user_time	8.09	8.09	8.09
Act 1	6.15	30.84	90.02
Act 2	1.67	7.26	28.26
Pool 1	7.26	7.26	7.26
Pool 2	2.14	2.14	2.14

classification. Nevertheless, it is important to acknowledge the limitations of our experiments, specifically the time consumed by the convolutional layer and the fully-connected layer. The GPU is exclusively utilized during the pre-training phase, while the CPU is employed for executing the PCCS. This is necessitated by the limitation of DT-PKC in supporting multiplication operations, which we address by implementing an interaction protocol instead of designing the underlying GPU code specific to the program implementation protocol.

C. Activation Function Comparison

To facilitate a more comprehensive comparison of the constructed function, we proceed to compare it with the common activation functions in Fig. 10. Initially, we compare the Taylor expansion with the Sigmoid function. Fig. 10(a) and (b) depict the four-series Taylor expansion of the Sigmoid function at integer points ranging from zero to ten. The Sigmoid function is represented by the line marked with a small circle, while the other lines in various colors represent the 4-series Taylor expansion at integer points ranging from zero to ten, denoted as $S_i(x), i = \{0, 1, 2, \dots, 10\}$. To provide additional information, supplementary local graphs have been included to illustrate the partial expansion of the Sigmoid function at integer points ranging from zero to ten. As depicted in Fig. 10(a) and (b), it is evident that the Sigmoid function's small circle at point i is precisely indicated on the corresponding image of $S_i(x), i = \{0, 1, 2, \dots, 10\}$.

By examining Fig. 10(c), it can be inferred that our proposed TanhTrans function exhibits a closer resemblance to the derivative of the Relu function in comparison to the Sigmoid function (excluding the 0 point). Fig. 10(d) displays the Relu, Softplus, and Tanhplus functions, wherein it is evident that our proposed Tanhplus function significantly outperforms in approximating the Relu function.

VIII. CONCLUSION

To address the privacy protection and nonlinear function fitting to cryptosystem questions in CNN classification schemes, we introduce a privacy-preserving PCCS with multiple

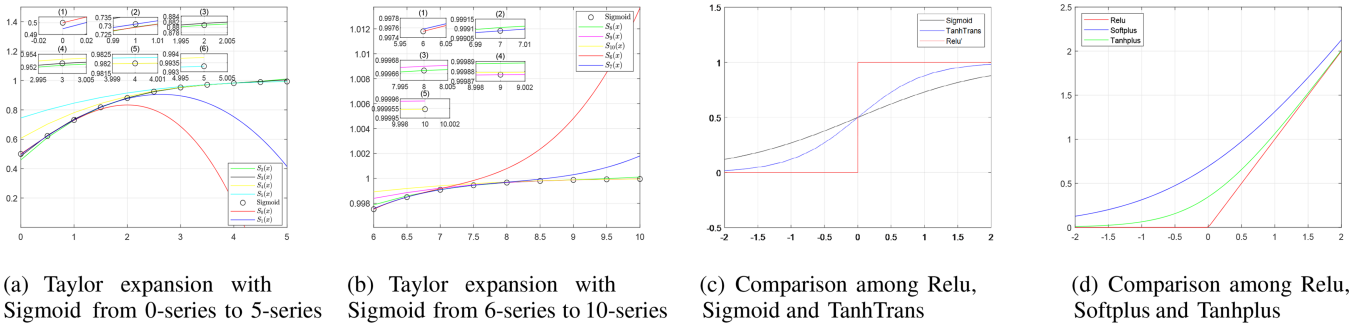


Fig. 10. Comparison among different functions.

keys. This scheme guarantees both the security and accuracy, while also providing theoretical and experimental support for lightweight and multiple users. We anticipate that this scheme can be effectively employed in the domain of image recognition for depth-specific CNNs. In subsequent research, our primary objective will be to enhance the efficiency of privacy-preserving CNN image classification on extensive datasets and more complex neural networks. Additionally, we aim to extend the application of this approach to diverse scenarios.

REFERENCES

- [1] Y. Chen, B. Wang, and Z. Zhang, "PDLHR: Privacy-preserving deep learning model with homomorphic re-encryption in robot system," *IEEE Syst. J.*, vol. 16, no. 2, pp. 2032–2043, Jun. 2022, doi: [10.1109/JSYST.2021.3078637](https://doi.org/10.1109/JSYST.2021.3078637).
- [2] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2012, pp. 1097–1105.
- [3] A. Graves, A. Mohamed, and G. Hinton, "Speech recognition with deep recurrent neural networks," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process.*, 2013, pp. 6645–6649.
- [4] I. Wallach, M. Dzamba, A. Heifets, I. Sutskever, and G. E. Hinton, "AtomNet: A deep convolutional neural network for bioactivity prediction in structure-based drug discovery," *Mathematische Zeitschrift*, vol. 47, no. 1, pp. 34–46, 2015.
- [5] Y. Taigman, M. Yang, M. Ranzato, and L. Wolf, "DeepFace: Closing the gap to human-level performance in face verification," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2014, pp. 1701–1708.
- [6] E. Grefenstette, P. Blunsom, N. De Freitas, and K. M. Hermann, "A deep architecture for semantic parsing," *Comput. Sci.*, vol. 30, no. 5, pp. 1–15, 2014.
- [7] B. Hitaj, G. Ateniese, and F. Pérez-Cruz, "Deep models under the GAN: Information leakage from collaborative deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 603–618.
- [8] G. Xu, H. Li, S. Liu, K. Yang, and X. Lin, "VerifyNet: Secure and verifiable federated learning," *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 911–926, 2020.
- [9] T. Li, J. Li, X. Chen, Z. Liu, W. Lou, and Y. Hou, "NPMML: A framework for non-interactive privacy-preserving multiparty machine learning," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 6, pp. 2969–2982, Nov./Dec. 2021.
- [10] Z. Tian, Y. Wang, Y. Sun, and J. Qiu, "Location privacy challenges in mobile edge computing: Classification and exploration," *IEEE Netw.*, vol. 34, no. 2, pp. 52–56, Mar./Apr. 2020.
- [11] C. Juvekar, V. Vaikuntanathan, and A. Chandrakasan, "GAZELLE: A low latency framework for secure neural network inference," in *Proc. 27th USENIX Secur. Symp.*, 2018, pp. 1651–1669.
- [12] H. Chen, W. Dai, M. Kim, and Y. Song, "Efficient multi-key homomorphic encryption with packed ciphertexts with application to oblivious neural network inference," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2019, pp. 395–412.
- [13] D. Rathee et al., "CrypTFLOW2: Practical 2-party secure inference," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2020, pp. 325–342.
- [14] X. Ma, J. Ma, H. Li, Q. Jiang, and S. Gao, "PDLML: Privacy-preserving deep learning model on cloud with multiple keys," *IEEE Trans. Services Comput.*, vol. 14, no. 4, pp. 1251–1263, Jul./Aug. 2021.
- [15] X. Zhang, X. Chen, J. Liu, and Y. Xiang, "DeepPAR and DeepDPA: Privacy preserving and asynchronous deep learning for industrial IoT," *IEEE Trans. Ind. Informat.*, vol. 16, no. 3, pp. 2081–2090, Mar. 2020.
- [16] L. T. Phong, Y. Aono, T. Hayashi, L. Wang, and S. Moriai, "Privacy-preserving deep learning via additively homomorphic encryption," *IEEE Trans. Inf. Forensics Security*, vol. 13, no. 5, pp. 1333–1345, May 2018.
- [17] E. Hesamifard, H. Takabi, and M. Ghasemi, "CryptoDL: Deep neural networks over encrypted data," *arXiv: 1711.05189*, 2017.
- [18] J. Chen, J. Zhou, Z. Cao, A. V. Vasilakos, X. Dong, and K. R. Choo, "Lightweight privacy-preserving training and evaluation for discretized neural networks," *IEEE Internet Things J.*, vol. 7, no. 4, pp. 2663–2678, Apr. 2020.
- [19] H. Chabanne, A. de Wargny, J. Milgram, C. Morel, and E. Prouff, "Privacy-preserving classification on deep neural network," *IACR Cryptol.*, vol. 2017, no. 35, pp. 1–18, 2017.
- [20] M. Baryalai, J. Jang-Jaccard, and D. Liu, "Towards privacy-preserving classification in neural networks," in *Proc. 14th Annu. Conf. Privacy, Secur. Trust*, 2016, pp. 392–399.
- [21] R. Xu, J. Joshi, and C. Li, "CryptoNN: Training neural networks over encrypted data," in *Proc. IEEE 39th Int. Conf. Distrib. Comput. Syst.*, 2019, pp. 1199–1209.
- [22] L. T. Phong and T. T. Phuong, "Privacy-preserving deep learning via weight transmission," *IEEE Trans. Inf. Forensics Security*, vol. 14, no. 11, pp. 3003–3015, Nov. 2019.
- [23] A. N. Khan, M. Y. Fan, A. Malik, and R. A. Memon, "Learning from privacy preserved encrypted data on cloud through supervised and unsupervised machine learning," in *Proc. 2nd Int. Conf. Comput., Math. Eng. Technol.*, 2019, pp. 1–5.
- [24] O.-A. Kwabena, Z. Qin, T. Zhuang, and Z. Qin, "MSCryptoNet: Multi-scheme privacy-preserving deep learning in cloud computing," *IEEE Access*, vol. 7, pp. 29344–29354, 2019.
- [25] E. Hesamifard, H. Takabi, M. Ghasemi, and E. Witchel, "Privacy-preserving machine learning as a service," in *Proc. Privacy Enhancing Technol.*, vol. 2018, no. 3, pp. 123–142, 2018.
- [26] J. Liu, M. Juuti, Y. Lu, and N. Asokan, "Oblivious neural network predictions via minionn transformations," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 619–631.
- [27] X. Liu, R. H. Deng, K. K. R. Choo, and J. Weng, "An efficient privacy-preserving outsourced calculation toolkit with multiple keys," *IEEE Trans. Inf. Forensics Security*, vol. 11, no. 11, pp. 2401–2414, Nov. 2016.
- [28] Y. Chen, Y. Ping, and Z. Zhang, "Privacy-preserving image multi-classification deep learning model in robot system of industrial IoT," *Neural Comput. Appl.*, vol. 33, pp. 4677–4694, 2020.
- [29] R. Bellafqira, G. Coatrieux, E. Genin, and M. Cozic, "Secure multilayer perceptron based on homomorphic encryption," in *Proc. Int. Workshop Digit. Watermarking*, 2018, pp. 322–336.
- [30] G. Wang, R. Lu, and C. Huang, "PSLP: Privacy-preserving single-layer perceptron learning for e-healthcare," in *Proc. 10th Int. Conf. Inf., Commun. Signal Process.*, 2015, pp. 1–5.

- [31] P. Mohassel and Y. Zhang, "SecureML: A system for scalable privacy-preserving machine learning," in *Proc. IEEE Symp. Secur. Privacy*, 2017, pp. 19–38.
- [32] B. D. Rouhani, M. S. Riazi, and F. Koushanfar, "DeepSecure: Scalable provably-secure deep learning," in *Proc. 55th Annu. Des. Automat. Conf.*, 2018, pp. 1–6.
- [33] R. Shokri and V. Shmatikov, "Privacy-preserving deep learning," in *Proc. 22nd ACM SIGSAC Conf. Comput. Commun. Secur.*, 2015, pp. 1310–1321.
- [34] R. Gilad-Bachrach, N. Dowlin, K. Laine, K. Lauter, and J. Wernsing, "CryptoNets: Applying neural networks to encrypted data with high throughput and accuracy," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 201–210.
- [35] M. Ribeiro, K. Grolinger, and M. A. M. Capretz, "MLaaS: Machine learning as a service," in *Proc. IEEE 14th Int. Conf. Mach. Learn. Appl.*, 2015, pp. 896–902.
- [36] E. Bresson, D. Catalano, and D. Pointcheval, "A simple public-key cryptosystem with a double trapdoor decryption mechanism and its applications," in *Proc. Int. Conf. Theory Appl. Cryptol. Inf. Secur.*, 2003, pp. 37–54.
- [37] R. Bost, R. A. Popa, S. Tu, and S. Goldwasser, "Machine learning classification over encrypted data," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2015, Art. no. 4325.
- [38] B. Wang, E. Tews, and S. Katzenbeisser, "A tale of two clouds: Computing on data encrypted under multiple keys," in *Proc. IEEE Conf. Commun. Netw. Secur.*, 2014, pp. 337–345.
- [39] A. Peter, E. Tews, and S. Katzenbeisser, "Efficiently outsourcing multiparty computation under multiple keys," *IEEE Trans. Inf. Forensics Security*, vol. 8, no. 12, pp. 2046–2058, Dec. 2013.
- [40] L. Xie, I. M. Baytas, K. Lin, and J. Zhou, "Privacy-preserving distributed multitask learning with asynchronous updates," in *Proc. 23rd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2017, pp. 1195–1204.
- [41] K. O'Shea and R. Nash, "An introduction to convolutional neural networks," *CoRR*, vol. abs/1511.08458, pp. 1–11, 2015.
- [42] N. Subramanian and A. Jeyaraj, "Recent security challenges in cloud computing," *Comput. Elect. Eng.*, vol. 71, pp. 28–42, 2018.
- [43] R. Canetti, "Universally composable security: A new paradigm for cryptographic protocols," in *Proc. 42nd Annu. Symp. Found. Comput. Sci.*, 2001, pp. 136–145.
- [44] P. Paillier, "Public-key cryptosystems based on composite degree residuosity classes," in *Proc. Int. Conf. Theory Appl. Cryptogr. Techn.*, 1999, pp. 223–238.
- [45] Z. Brakerski, C. Gentry, and V. Vaikuntanathan, "(Leveled) fully homomorphic encryption without bootstrapping," in *Proc. 3rd Innov. Theor. Comput. Sci. Conf.*, 2012, pp. 309–325.
- [46] G. Xu, H. Li, Y. Zhang, S. Xu, J. Ning, and R. H. Deng, "Privacy-preserving federated deep learning with irregular users," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 2, pp. 1364–1381, Mar./Apr. 2022.



Baocang Wang received the BS and MS degrees in mathematics from Xidian University, in 2001 and 2004, respectively, and the PhD degree in cryptography from Xidian University, in 2006. He is a professor with the School of Telecommunications Engineering, Xidian University. His main research interests include public key cryptography, encryption data processing, data mining security.



Yange Chen received the BS and MS degrees in computer application technology from Henan Polytechnic University, in 2006 and 2008, respectively, and the PhD degree in cryptography from Xidian University, in 2022. She is an associate professor with the School of Information Engineering, Xuchang University. Her main research interests include deep learning security, Internet of Things security, homomorphic encryption.



Furong Li received the BS degree in telecommunication from the Hainan University, Hainan, China, in 2018 and the MS degree in electronic and communication engineering with the School of Telecommunications Engineering, Xidian University, Xi'an, China, in 2021. She is currently working with the 20th Research Institute of China Electronics Technology Group Corporation. Her main research interests include deep learning security, convolutional neural network, homomorphic encryption.



Jian Song received the MS degree in electronic and communication engineering with the School of Telecommunications Engineering, Xidian University, Xi'an, China. He is currently working with Zhongxing Telecommunications Equipment Corporation. He specializes in Python programming and Pytorch framework. His main research interests include convolutional neural network, computer vision and object detection.



Technical Committee). He is the Winner of 2016–17 Excellence in Teaching Award, FCS, UNB.

Rongxing Lu (Fellow, IEEE) is currently an associate professor with the Faculty of Computer Science (FCS), University of New Brunswick (UNB), Canada. His research interests include applied cryptography, privacy enhancing technologies, and IoT-Big Data security and privacy. He has published extensively in his areas of expertise, and was the recipient of 9 best (student) paper awards from some reputable journals and conferences. Currently, he serves as the vice-chair (Conferences) of IEEE ComSoc CIS-TC (Communications and Information Security



plementation of privacy-preserving technologies.

Pu Duan received the PhD degree from Texas A & M University, in 2011. He has been a twenty-year veteran on cryptography, information security and networking security. He has published 20+ papers on areas of cryptography, networking security and system security. He joined Cisco after obtaining his PhD degree. At Cisco he led the research and development of new cryptographic algorithms for TLS.13 on cisco firewall product. Currently he works in Secure Collaborative Lab (SCI) lab, Ant Group as a senior staff engineer, leading the team on research and



the Harbin Institute of Technology. He has authored more than 200 journal and conference papers. His research interests include computer networks and cyberspace security. His research works has been supported by the National Natural Science Foundation of China, National Key research and Development Plan of China, and National High-tech R&D Program of China (863 Program). He also served as a member, chair, and general chair of a number of international conferences. He is a distinguished member of the China Computer Federation.

Zhihong Tian (Senior Member, IEEE) received the BS, MS, and PhD degrees in computer science and technology from the Harbin Institute of Technology, Harbin, China, in 2001, 2003, and 2006, respectively. He is currently a professor, and dean, with the Cyberspace Institute of Advanced Technology, Guangzhou University, Guangdong Province, China. He is honored as Pearl River Scholar in Guangdong Province. He is also a part-time professor with Carlton University, Ottawa, Canada. Previously, he served in different academic and administrative positions with